

**Séance 3**  
**Attaque Man-In-The-Middle (MITM) d'un service Secure Shell Protocol (SSH)**  
**Mise en place de contre-mesures**

**Email pour le rapport :** romain.dulout@u-bordeaux.fr

**A. Introduction - Retour sur le chiffrement et nouveaux éléments**

**Q1. Petit retour sur le chiffrement symétrique et asymétrique :** Lors d'un échange sécurisé sur HTTPS, pourquoi utilise-t-on un chiffrement asymétrique au départ, puis un chiffrement symétrique pour la suite de la communication ?

Quelques éléments supplémentaires pour mieux comprendre le fonctionnement et les règles régissant les clés de chiffrement.

**Q2. Qu'en est-il des règles de longueur de clés ?**

- Sont-elles régulées ?
- Sont-elles les mêmes pour des algorithmes de chiffrement symétrique et de chiffrement asymétrique ? (Source : <https://www.bibmath.net/crypto/index.php?action=affiche&quoi=chasseur/key>)
- Donnez trois noms d'algorithmes de chiffrement symétrique et de chiffrement asymétrique (source: <https://blog.mailfence.com/fr/difference-chiffrement-symetrique-asymetrique/>)

**Q3. Et le hachage dans tout ça, de quoi s'agit-il ?**

- A quoi servent les fonctions de hachage ?
- Que sont les problèmes de collisions ?

(Sources : <https://www.digikey.fr/fr/articles/iot-security-fundamentals-part-1-using-cryptography>  
<https://next.innk/10581/103436-sha-1-chercheurs-prouvent-exploitation-collisions-dans-attaque/> )

**B. Mise en place d'une authentification par clés de chiffrement**

Lorsque vous générez une paire de clés de chiffrement (publique/privée), vous devez vous assurer que celle-ci n'est pas prédictible par un attaquant. Ceci peut-être mesuré grâce à l'entropie.

**Q4. Qu'est ce que l'entropie ? Pourquoi cet élément est-il essentiel ?**

(Source : Point 2 du fichier Ressources)

Note : Dans un contexte professionnel, la génération de clés de chiffrement doit être réalisée sur une machine physique. De plus, cette machine doit disposer de plusieurs sources d'entropie indépendantes. La génération de clés sur des machines virtuelles conduit à une baisse trop importante de l'entropie et donc à une mauvaise qualité des clés créées.

Quelques informations complémentaires à ce sujet :  
<https://www.linkedin.com/pulse/un-peu-dentropie-sous-linux-stephane-n/?originalSubdomain=fr>

L'authentification par mot de passe est considérée comme un mécanisme plutôt faible. Elle peut être soumise à différents types d'attaques comme les attaques par dictionnaire, par force brute ou par Man In The Middle comme nous l'avons vu précédemment. Ainsi lorsque l'on configure un service SSH, il est recommandé de mettre en œuvre une authentification plus robuste par double facteur d'authentification (2FA) ou par clés de chiffrement.

#### **Q5. Qu'est ce qu'une authentification par double facteur d'authentification ?**

(Source : <https://www.varonis.com/fr/blog/authentification-a-deux-facteurs>)

- Éditez *en mode root* le fichier de configuration serveur (*/etc/ssh/sshd\_config*) et autorisez ce mécanisme de connexion en décommentant les lignes suivantes :

```
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

- Redémarrez ensuite le service : `sudo systemctl restart ssh`

Les manipulations sur le serveur sont maintenant terminées. Quittez éventuellement la connexion SSH sur le serveur (si vous y accédez ainsi) puis basculez sur le client.

- Sur la machine cliente, l'utilisateur `etusio` va devoir générer sa clé publique, et sa clé privée afin de pouvoir s'authentifier sur le serveur OpenSSH :

```
etusio@clissh:~$ ssh-keygen -b 256 -t ecdsa
```

#### **Q6. A quoi correspondent "ecdsa" et 256 dans cette ligne de commande ?**

Le générateur de clés va en générer deux, une clé publique et une clé privée. Il va placer la clé privée dans un endroit qui, par défaut, est `$HOME/.ssh/id_ecdsa`

- Appuyez sur Entrée pour accepter la localisation de la clé par défaut. `ssh-keygen` demande ensuite une phrase de chiffrement (ou « passphrase », équivalent d'un mot de passe, mais sous forme de phrase).
- Tapez `TPMITM` comme phrase de chiffrement et confirmez-la. Vous obtenez l'empreinte de votre clé (« key fingerprint ») et une « randomart image ».

#### **Q7. Pourquoi l'algorithme ECDSA a été préféré à l'algorithme RSA lors de la génération de la paire de clés ? Quels sont les avantages de chacun de ces**

## algorithmes ? Pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?

(Source : <https://sectigostore.com/blog/ecdsa-vs-rsa-everything-you-need-to-know/> )

Que faire maintenant de la paire de clés ?

Si l'on récapitule, ssh-keygen a généré deux clés :

- Une clé privée qui est `$HOME/.ssh/id_ecdsa` à laquelle vous seul devez avoir accès
- Une clé publique qui est `$HOME/.ssh/id_ecdsa.pub`, qui peut être connue par tout le monde.

### Q8. Quel serait le risque si quelqu'un d'autre avait accès à votre clé privée ?

Sur le serveur, vous devez maintenant autoriser explicitement votre compte présent sur la machine cliente à accéder via ssh à celui-ci. Pour ce faire, vous devez ajouter dans le répertoire `.ssh` de l'utilisateur qui sera choisi pour se connecter (exemple : `/home/etusio/.ssh`) la clé publique générée précédemment (`id_ecdsa.pub`) dans un fichier `authorized_keys`. La méthode la plus simple est d'utiliser la commande `ssh-copy-id` qui copiera la clé dans un fichier `authorized_keys` à l'emplacement défini dans votre chemin (`~/.ssh/`), même si celui-ci n'existe pas au préalable.

- Pour ce faire, utilisez la ligne suivante :  
`ssh-copy-id -i <Chemin/Fichier> <utilisateur>@<IP ou non du serveur>`

Note : Si vous rencontrez des erreurs, celles-ci seront dues aux attaques que nous avons mené durant la séance précédente à l'encontre du serveur. Il pourrait vous être nécessaire de retirer le serveur enregistré de la liste des known hosts pour résoudre ce problème (cf. Séance 2).

- Une fois l'ajout effectué, depuis le poste client, relancez une connexion ssh au serveur. La phrase de chiffrement vous est bien demandée, cela vous permet de vérifier que la connexion avec clé publique est fonctionnelle.
- Sur le serveur Debian, désactivez l'authentification par mot de passe pour juste conserver celle par clés dans le fichier de configuration `/etc/ssh/sshd_config`. Pour ce faire, dans le fichier de configuration, décommentez la directive `PasswordAuthentication` puis affecter le paramètre `no`.

-----  
Note : Si vous désactivez l'authentification par mot de passe, vous devez vous assurer que l'authentification par clé est opérationnelle sous peine de ne plus du tout pouvoir accéder à la machine distante !  
-----

- Redémarrez le service SSH pour que la modification du fichier soit prise en compte :

Nous allons maintenant vérifier que ssh se préoccupe de la sécurité de vos clés.

- Sur la machine cliente clissh.local.sio.fr, modifiez les droits du fichier ~/.ssh/id\_ecdsa (droits initiaux : 600 etusio:etusio) qui contient votre clé privée en 644 et connectez vous sur le serveur distant srvssh.local.sio.fr qui contient la clé publique.

#### Q9. Que se passe-t-il ? Pourquoi ?

- Rétablissez les droits de ~/.ssh/id\_ecdsa sur le poste client
- Sur le serveur distant, affichez les droits d'accès appliqués au fichier ~/.ssh/authorized\_keys (ls -al ou autre). Puis, modifiez les droits de ~/.ssh/authorized\_keys en 666. Déconnectez vous et reconnectez vous.

**Q10. Vérifiez si un changement est apparu ou non et tentez d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite). En analysant la connexion par clés, proposez une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquez ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.**

#### C. Utilisation de ssh-agent

#### Q11. Qu'est ce qu'un agent ssh ? Pourquoi peut-il être utile ?

(Source : <https://smallstep.com/blog/ssh-agent-explained/>)

- Sur le client, tapez les commandes suivantes, pour mettre en place le ssh-agent. Par défaut sous Debian, le bureau GNOME intègre un trousseau de clés qui fonctionne en adéquation avec ssh-agent et qui permet d'éviter de saisir à chaque fois des mots de passe.

```
etusio@clissh:~$ exec ssh-agent $SHELL
```

Note : L'utilisation de la variable \$SHELL permet d'exécuter l'agent ssh dans l'interpréteur en cours d'utilisation.

```
etusio@clissh:~$ ssh-add
Enter passphrase for ~/.ssh/id_ecdsa:
Identity added: ~/.ssh/id_ecdsa (~/.ssh/id_ecdsa)
```

- Entrez la *passphrase* configurée au préalable. Pendant toute la durée de la connexion, il est possible d'avoir accès à la machine distante sans avoir à taper un mot de passe. Si le processus ssh-agent disparaît, il sera nécessaire de retaper le mot de passe.

**Q12. Suite à la mise en place de cette authentification par clés de chiffrement, en vous appuyant sur la première séance, tentez à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?**

- Effacez à nouveau le contenu du fichier `/home/etusio/.ssh/known_hosts` avant de poursuivre la suite du TP et pensez également à arrêter l'attaque si cela n'a pas encore été fait.

#### D. Faciliter la vérification de l'identité du serveur SSH avant la première connexion

OpenSSH adopte par défaut un modèle de sécurité appelé Trust On First Use (TOFU) (cf. Ressource de la Séance 1). Lors de la première connexion et à défaut de pouvoir authentifier l'hôte, ssh demande confirmation à l'utilisateur qu'il s'agit bien de la bonne clé via son empreinte. Si l'utilisateur confirme que l'empreinte est bonne, ssh procèdera à son enregistrement afin de permettre sa vérification lors des visites suivantes.

Dès lors, il existe plusieurs manières de contrôler l'identité du serveur :

1. En s'assurant que l'empreinte de la clé présentée est identique à celle présente sur le serveur (`ssh-keygen -l`) ;
2. En ajoutant la clé manuellement au préalable dans le fichier `known_hosts` du poste client.

Comme vous pouvez le constater, ces méthodes sont assez lourdes à mettre en œuvre lorsque l'on administre plusieurs équipements à l'aide de ce protocole. Le mécanisme SSHFP permet d'offrir plus de souplesse dans cette étape capitale de vérification.

#### Q13. De quoi s'agit-il à votre avis ? Expliquez l'intérêt de cette démarche.

(Source : Ressources présentes dans ce dossier)

- Tout d'abord, assurez-vous manuellement que l'identité du serveur (l'empreinte de la clé présentée) lors de la première connexion est bien celle de la clé publique présente sur le serveur, comme le montre l'exemple ci-dessous :

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:IP1xEKxsYHPP3i7iMiZZXLYUoW9viLwSf39MNoWIM4 root@srvssh (ECDSA)
```

- A comparer à l'empreinte affichée sur le client :

```
ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:IP1xEKxsYHPP3i7iMiZZXLYUoW9viLwSf39MNoWIM4.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'srvssh.local.sio.fr, 192.168.56.10' (ECDSA) to the list of known hosts.
etusio@srvssh.local.sio.fr's password:
```

- Pour obtenir les enregistrements SSHFP à ajouter dans votre fichier de zone DNS, voici la commande à taper sur le serveur (Important, c'est le nom d'hôte du serveur qui doit être saisi) :

```
etusio@srvssh:~$ sudo ssh-keygen -r srvssh
srvssh IN SSHFP 1 1 6fa8aeb8227f09bc4f8c8afe08245a8c7718d324
srvssh IN SSHFP 1 2 ef3c9989952d8591cc761f1b26a658354a93230dae730edbe42732bf0a1219a8
srvssh IN SSHFP 3 1 4c16f0e9298469630445a24c57c342b15540253f
srvssh IN SSHFP 3 2 94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30da1694ce
srvssh IN SSHFP 4 1 7ef6631af09e0585ff8c3169255189195f7b7ddf
srvssh IN SSHFP 4 2 88efa67bb697dafdde1c18bee5c76218e8c78803bfa6b3e093c260cf56bd9f6e
```

- Ajoutez ces nouveaux enregistrements à la fin du fichier de zone présent sur le serveur SSH/DNS.

```
etusio@srvssh:~$ sudo edit /var/named/db.local.sio.fr
```

- Rechargez le service :

```
etusio@srvssh:~$ sudo systemctl reload bind9
```

- Sur la machine cliente, si vous souhaitez que cette vérification se fasse au moment de la connexion, voici la commande à utiliser :

```
etusio@clissh:~$ ssh -o VerifyHostKeyDNS=true etusio@srvssh.local.sio.fr
```

- Si vous souhaitez que cette vérification ait lieu à chaque nouvelle connexion depuis votre poste client, vous pouvez rajouter cette option dans le fichier de configuration du client (/etc/ssh/ssh\_config) :

```
VerifyHostKeyDNS yes
```

#### **D. Durcissement des algorithmes de chiffrement utilisés entre le client et le serveur SSH**

Lorsque l'on implémente des protocoles chiffrés comme https ou ssh, il est primordial de respecter l'état de l'art en matière de choix des algorithmes utilisés.

**Q14. Expliquez dans une définition succincte ce qu'est l'état de l'art dans le domaine de la cybersécurité.**

**Q15. Définissez ce qu'est le principe de Kerckhoffs. Selon ce principe, pourquoi est-il pertinent de choisir des algorithmes cryptographiques connus et respectant l'état de l'art ?**

(Source potentielle : Ressources Séance 2)

Des agences de sécurité nationale comme l'ANSSI en France ou les experts en sécurité informatique de grandes entreprises comme Mozilla publient régulièrement les algorithmes à privilégier lorsque l'on utilise OpenSSH ou TLS. Pour empêcher l'usage d'algorithmes de chiffrement dépréciés, il est nécessaire d'éditer le fichier de configuration du serveur SSH, d'ajouter les lignes suivantes puis de le redémarrer.

**Q16. A quoi semblent correspondre ces lignes ?**

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
```

- Lignes à ajouter :

-----

KexAlgorithms

curve25519-sha256@libssh.org,ecdh-sha2-nistp521,ecdh-sha2-nistp384,ecdh-sha2-nistp256,diffie-hellman-group-exchange-sha256

Ciphers

chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr

MACs

hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-512,hmac-sha2-256,umac-128@openssh.com

-----

```
etusio@srvssh:~$ sudo systemctl restart sshd
```